COMP 303 Final Review

Winter 2024
Prof. Jin L.C. Guo

By Billy Exarhakos

Quick Overview of Pre-Midterm Material

- Flyweight
- Singleton
- Iterator
- Design By Contract
- Tests
- Diagrams

Flyweight

Purpose :

Used to avoid duplicate creation of objects.

Steps :

1. Make constructor private
2. Add static data structure to keep track of created objects
3. Add static function to get objects

Singleton

Purpose :

Used to ensure only one instance of a class is ever created.

Steps :

1. Make constructor private
2. Add static instance field to class
3. Add static getInstance() method

Iterator

Purpose :

The client can traverse through objects without knowing how they are stored internally

Steps :

1. Implement Iterable<Type>
2. Override iterator(), typically will return the iterator of your data structure (i.e. aList.iterator()).

Design By Contract

Purpose :

Explicitly states preconditions that the caller of a function must abide by, as well as postconditions

Steps :

1. State your contract above the method using @pre, @post and @inv
2. Assert your contract in the method using assert statements

Testing

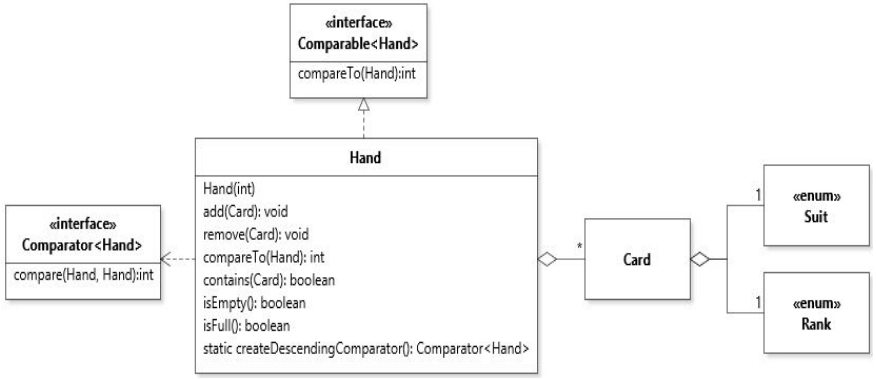
Purpose :

To make sure that the code's behaviour is as we expect it to be.

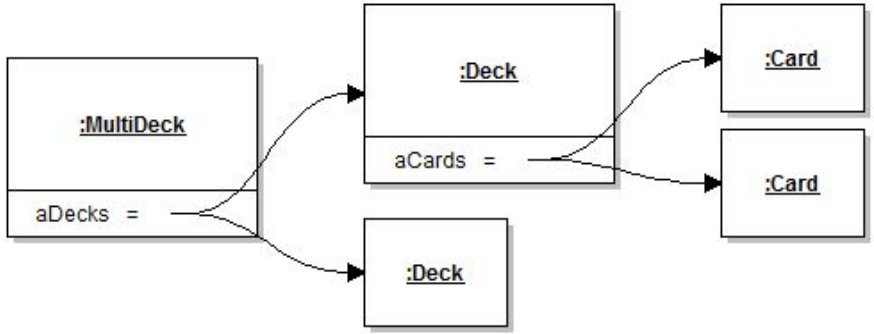
Steps :

1. Annotate with @test
2. Keep tests atomic (don't make one giant test case)
3. Check the behaviour using JUnit assert methods (assertEquals(), assertThrows()...). Do NOT use assert statements.

Diagrams

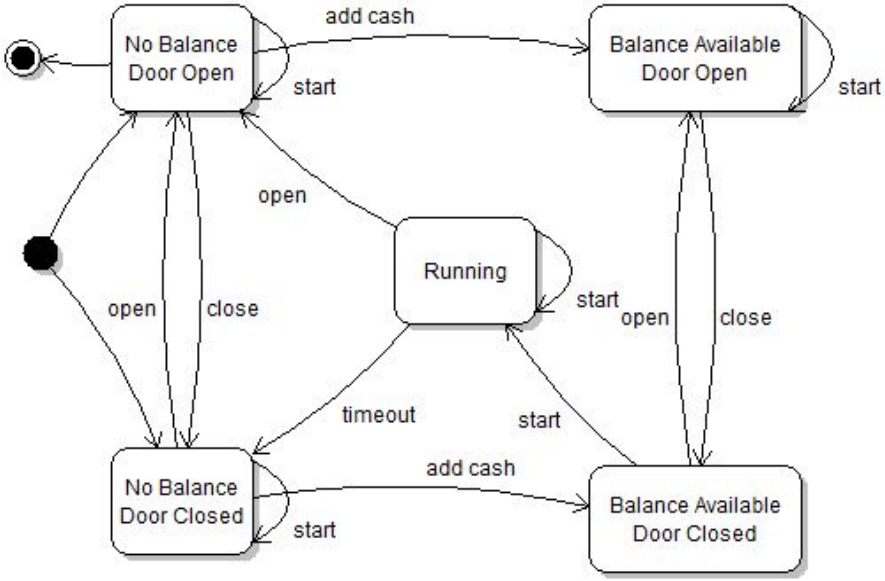


Class Diagram : Shows relationships between classes

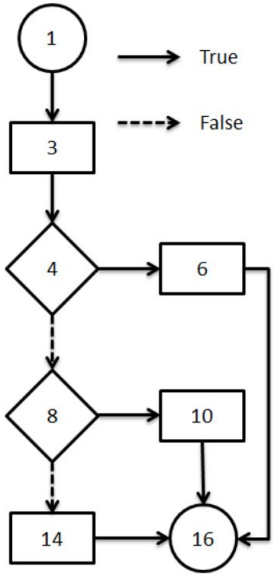


Object Diagram : Shows objects and data values during runtime

Diagrams



State Diagram : Shows the different states and transitions of a software component



Control Flow Graph : Shows the different paths of execution of a piece of code

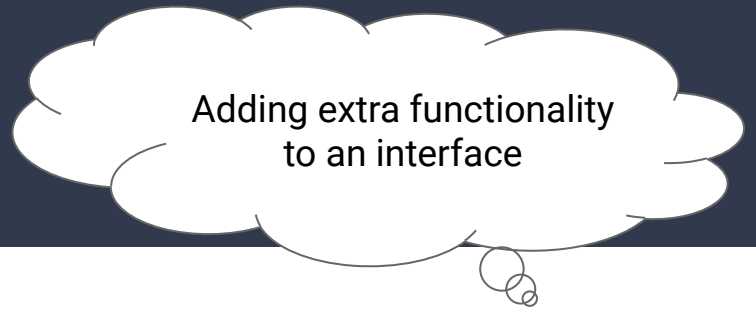
Now onto the post-midterm material...

Question 1

You are a grocery store owner and your job is to keep track of the produce you sell. To do this, you have a Produce interface which specifies two methods : getName() and getPrice(). You also have two concrete classes already : an Apple class and a Pepper class.

Add a class which represents exotic produce. Exotic produce is produce that is imported from another country and is therefore more expensive. The price of exotic produce is the regular price increased by 20% and its name should have the produce name followed by “imported from : ” and then whichever country it was imported from.

Question 1 : Decorator



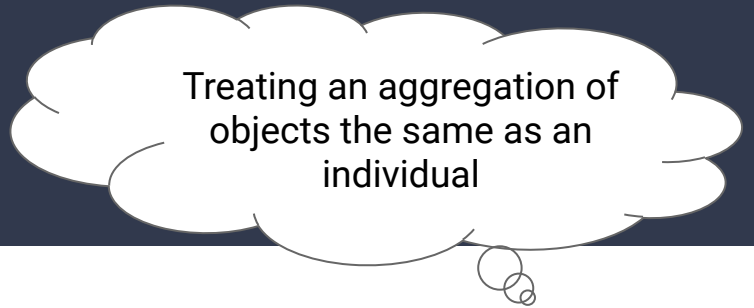
You are a grocery store owner and your job is to keep track of the produce you sell. To do this, you have a Produce interface which specifies two methods : getName() and getPrice(). You also have two concrete classes already : an Apple class and a Pepper class.

Add a class which represents exotic produce. Exotic produce is produce that is imported from another country and is therefore more expensive. The price of exotic produce is the regular price increased by 20% and its name should have the produce name followed by “imported from : ” and then whichever country it was imported from.

Question 2

Specify a class which represents a produce basket. A produce basket is made up of many different types of produce, and its price is the sum of all of the prices of the produce it contains. Its name should also include all the produce items inside of the basket.

Question 2 : Composite



Specify a class which represents a produce basket. A produce basket is made up of many different types of produce, and its price is the sum of all of the prices of the produce it contains. Its name should also include all the produce items inside of the basket.

Question 2b : Sequence Diagram

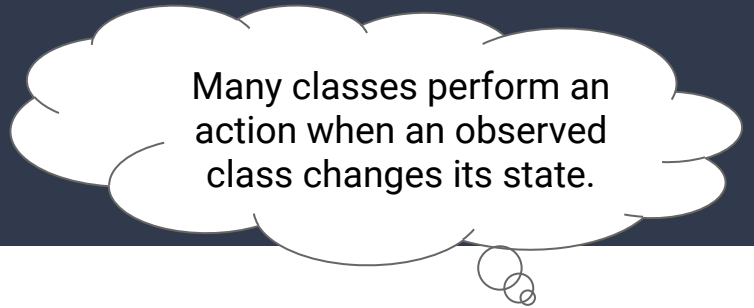
Draw the sequence diagram when calling getName on a fruit basket containing:

- An apple with name: “Honeycrisp” and price: 2
- A pepper with name: “Habanero” and price: 1
- An exotic apple from Japan with name: “Fuji” and price: 5

Question 3

The provided WeatherTracker class keeps track of the current temperature, humidity and air pressure. Modify the code so that any time the weather changes, a TemperatureTracker class prints the new temperature and a HumidityTracker class prints the new humidity. Your design should use the pull data-flow strategy.

Question 3 : Observer

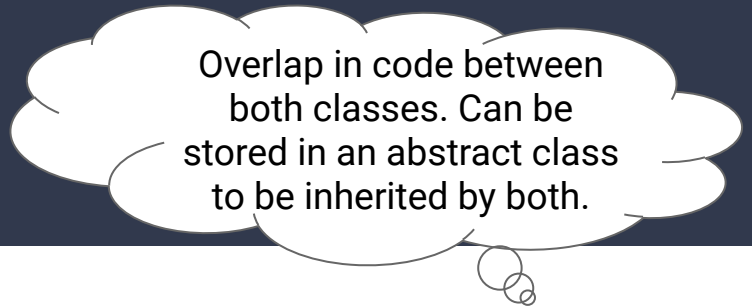


The provided WeatherTracker class keeps track of the current temperature, humidity and air pressure. Modify the code so that any time the weather changes, a TemperatureTracker class prints the new temperature and a HumidityTracker class prints the new humidity. Your design should use the pull data-flow strategy.

Question 4

You are given a Car class and a Truck class. Both classes keep track of their license plate and their model name. However, cars keep track of the number of seats, while trucks keep track of their associated company. Modify the code so that it follows better design principles and reduces code repetition.

Question 4 : Inheritance



You are given a Car class and a Truck class. Both classes keep track of their license plate and their model name. However, cars keep track of the number of seats, while trucks keep track of their associated company. Modify the code so that it follows better design principles and reduces code repetition.